



HashiCorp
Vault

Some content has been intentionally redacted (proprietary information and screenshots).

This documentation was created using docs-as-code, with a Static Site Generator (SSG) seamlessly converting Markdown content into HTML. The SSG took charge of formatting, while shortcodes were used for callouts and code blocks. Links between internal applications and documentation have been removed.

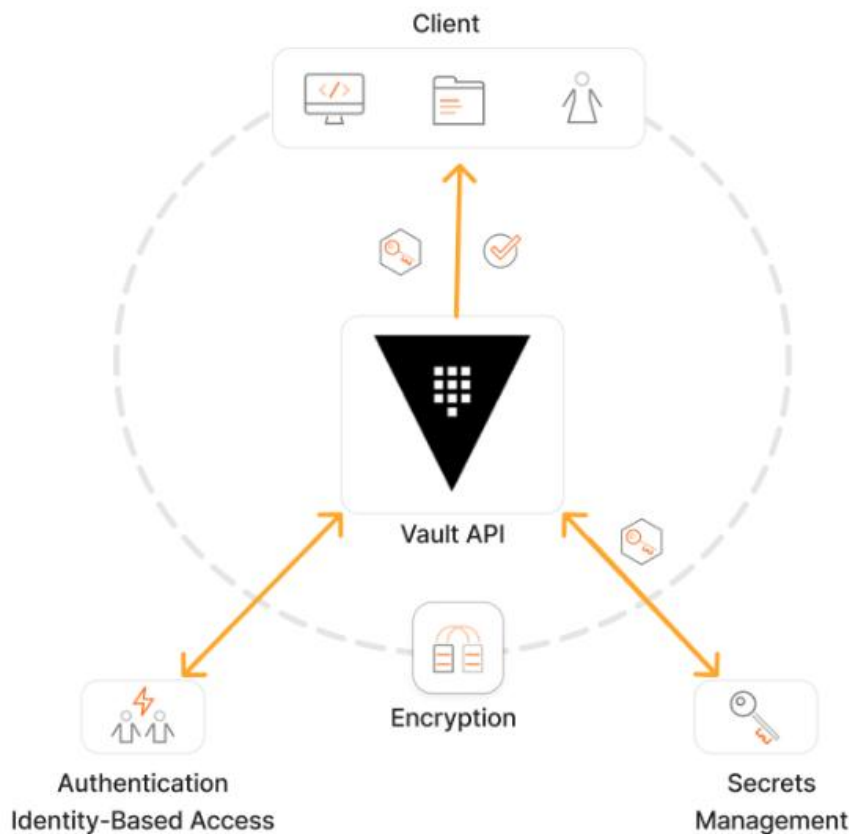
This snippet is a glimpse of an internal product's landing page; the full document covers APIs, Use Cases (with mermaid diagrams), Tutorials and Guides, Vault Integration Mechanisms, and Support.

HashiCorp Vault Overview

HashiCorp Vault (HCV) is an identity-based secrets and encryption management system. Vault validates and authorizes clients—such as users, machines, and applications—before granting access to secrets, cryptographic keys, or other sensitive data.

All interactions with HashiCorp Vault occur through its HTTP API. Most requests require a Vault token, which is obtained by authenticating through one of Vault's supported authentication methods, such as OIDC, LDAP, or username and password.

The following sections provide an overview of how secrets and encryption management are handled using HashiCorp Vault.



Use Cases

Secrets Management

Centrally store, access, and distribute secrets across applications, systems, and infrastructure in a consistent and secure manner.

Identity-Based Access

Authenticate to and access cloud platforms, systems, and endpoints using trusted identities, rather than long-lived static credentials.

Data Encryption and Tokenization

Secure application data using centralized encryption and tokenization workflows, while allowing encrypted data to reside in untrusted or semi-trusted systems outside of Vault.

Key Management

Standardize key distribution workflows and lifecycle management across multiple key management software providers and platforms.

How It Works

Vault uses tokens to represent authenticated clients. Each token is associated with one or more policies, which define what paths and operations the client is allowed to access. Policies in Vault are path-based and constrain which actions a client can perform on specific API endpoints.

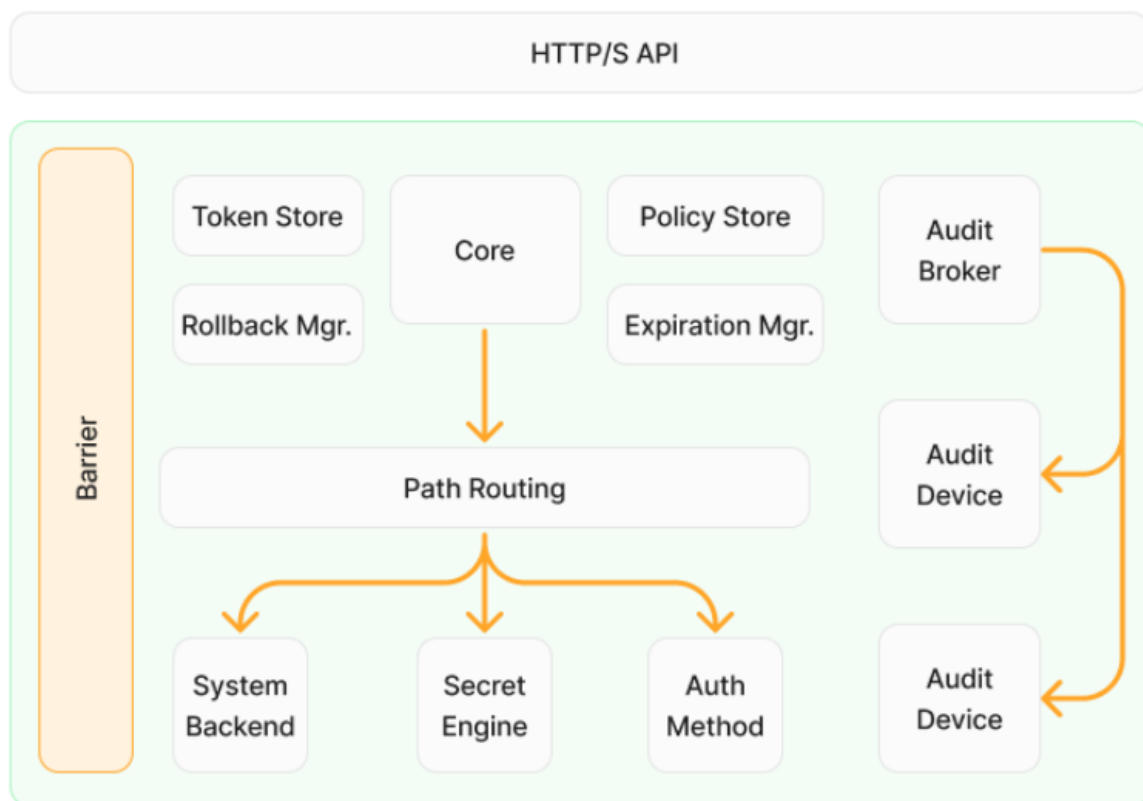
Clients can be issued tokens manually, or they can authenticate to Vault and receive a token dynamically through an authentication method.

Workflow

The core Vault workflow consists of three stages:

1. **Authenticate:** A client authenticates using an authentication method (for example, OIDC, LDAP, or username and password). Vault validates the client against the configured identity provider and issues a token.
3. **Authorize:** For each request, Vault evaluates the token's associated policies to determine whether the requested path and operation are permitted.
4. **Access:** If authorized, Vault processes the request and grants access to secrets, keys, or encryption capabilities. The client can reuse the same Vault token for subsequent operations until it expires or is revoked.

The following diagram illustrates the HashiCorp Vault request flow (developer view), showing how requests are authenticated, authorized, routed, audited, and persisted.



Components

The following terms and descriptions correspond directly to the elements shown in the Vault request flow diagram and are intended to help developers understand how requests are processed within Vault.

HTTP/S API

The HTTP API is the entry point to Vault. All client interactions with Vault occur through this API over HTTPS.

Core

The Vault Core coordinates request handling and manages internal subsystems such as authentication, policy enforcement, token management, auditing, and secret lifecycle operations.

Barrier

The barrier is cryptographic steel and concrete around Vault. All data that flows between Vault and the storage backend passes through the barrier. The barrier ensures that only encrypted data is written to storage, and that data is verified and decrypted when it is read back into Vault.

Path Routing

Vault routes requests based on their API path prefix to the appropriate internal component, such as a secrets engine, auth method, or system backend.

Secrets Engine

A secrets engine is responsible for managing secrets. Simple secrets engines return the same secret when queried. Other secrets engines can dynamically generate secrets each time they are accessed, allowing for fine-grained access control, revocation, and lifecycle management.

Auth Method

An auth method is used to authenticate users or applications that connect to Vault. After successful authentication, Vault associates the client with one or more policies and returns a client token that can be used for future requests.

System Backend

The system backend exposes internal Vault endpoints used to manage configuration, policies, authentication methods, audit devices, and other operational settings.

Token

A token (also referred to as a Vault token) is conceptually similar to a session cookie on a website. Once a client authenticates, Vault issues a token that represents the client's identity and is used to enforce the associated policies on subsequent requests.

Audit Broker

Vault internally coordinates audit logging and forwards request and response information to all configured audit devices.

Audit Device

An audit device records detailed logs of Vault requests and responses. Every authenticated request to Vault and every response from Vault is sent to all configured audit devices, enabling integration with one or more audit logging destinations.

Secret

A secret is any value returned by Vault that contains confidential or cryptographic material, such as credentials, encryption keys, or certificates.

Not everything returned by Vault is a secret, for example, system configuration, status information, and policies are not considered secrets.

Tutorials and Guides

Here is a snapshot of the guides created for this product. Full details have been omitted, as they contain proprietary screenshots and code snippets.

